Section 1: Prompt
Write a recursive object-oriented program in C++ to find the solution to the five Sudoku puzzle files using a depth-first search with backtracking to brute-force the solution.
The output for each solution, should include:
-The puzzle file name (e.g., puzzle1.txt)
-The puzzle from the file printed out.
-The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
The five puzzle files are named puzzle1.txt through puzzle5.txt and will be in the same folder as the code. Each contains a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.

Section 2: AI’s used and method of access
ChatGPT and Gemini, both were accessed via their respective publicly available websites.

Section 3: Generated Code
Gemini:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuSolver {
private:
std::vector<std::vector<char>> board;

// Check if placing character 'c' at board[row][col] is valid
bool isValid(int row, int col, char c) const {
for (int i = 0; i < 9; ++i) {
// Check row and column
if (board[row][i] == c) return false;
if (board[i][col] == c) return false;
}

// Check 3x3 subgrid
int startRow = row - row % 3;
int startCol = col - col % 3;
for (int i = 0; i < 3; ++i) {
for (int j = 0; j < 3; ++j) {
if (board[startRow + i][startCol + j] == c) return false;
}
}
return true;
}

// Recursive DFS backtracking function to find all solutions
bool solveRecursive(int row, int col, int& solutionCount) {
// If we have crossed the last row, a complete solution is found
if (row == 9) {
```

```
solutionCount++;
std::cout << "Solution " << solutionCount << ":\n";
printBoard();
std::cout << "\n";
return false; // Return false to force backtracking and search for alternative solutions
}

// Move to the next row if column reaches 9
if (col == 9) {
return solveRecursive(row + 1, 0, solutionCount);
}

// If cell is already filled, skip to the next column
if (board[row][col] != '_') {
return solveRecursive(row, col + 1, solutionCount);
}

// Try digits '1' through '9'
for (char c = '1'; c <= '9'; ++c) {
if (isValid(row, col, c)) {
board[row][col] = c;
solveRecursive(row, col + 1, solutionCount);
board[row][col] = '_'; // Backtrack
}
}

return false;
}

public:
SudokuSolver() : board(9, std::vector<char>(9, '_')) {}

// Load puzzle from file
bool loadFromFile(const std::string& filename) {
std::ifstream file(filename);
if (!file.is_open()) {
return false;
}
for (int i = 0; i < 9; ++i) {
for (int j = 0; j < 9; ++j) {
file >> board[i][j];
}
}
return true;
}

// Print the Sudoku board with clear grid formatting
void printBoard() const {
for (int i = 0; i < 9; ++i) {
if (i > 0 && i % 3 == 0) {
```

```
std::cout << "------+-------+------\n";
}
for (int j = 0; j < 9; ++j) {
if (j > 0 && j % 3 == 0) {
std::cout << "| ";
}
std::cout << board[i][j] << " ";
}
std::cout << "\n";
}
}

// Driver method to process a puzzle file
void solvePuzzleFile(const std::string& filename) {
std::cout << "=====================================\n";
std::cout << "Puzzle File: " << filename << "\n";
std::cout << "=====================================\n";

if (!loadFromFile(filename)) {
std::cout << "Error: Could not open file " << filename << ".\n\n";
return;
}

std::cout << "Original Puzzle:\n";
printBoard();
std::cout << "\n";

int solutionCount = 0;
solveRecursive(0, 0, solutionCount);

if (solutionCount == 0) {
std::cout << "No solution found.\n\n";
} else {
std::cout << "Total solutions found: " << solutionCount << "\n\n";
}
}
};

int main() {
SudokuSolver solver;

// Process puzzle1.txt through puzzle5.txt
for (int i = 1; i <= 5; ++i) {
std::string filename = "puzzle" + std::to_string(i) + ".txt";
solver.solvePuzzleFile(filename);
}

return 0;
}
```

ChatGPT:

```
#include <iostream>
#include <fstream>
#include <string>
#include <cctype>
using namespace std;

class SudokuSolver {
private:
int grid[9][9];
int solutionCount;

// Check whether a number can be placed in a cell.
bool isSafe(int row, int col, int num) {
// Check the row.
for (int c = 0; c < 9; c++) {
if (grid[row][c] == num)
return false;
}

// Check the column.
for (int r = 0; r < 9; r++) {
if (grid[r][col] == num)
return false;
}

// Check the 3x3 subgrid.
int startRow = (row / 3) * 3;
int startCol = (col / 3) * 3;

for (int r = startRow; r < startRow + 3; r++) {
for (int c = startCol; c < startCol + 3; c++) {
if (grid[r][c] == num)
return false;
}
}

return true;
}

// Print the current Sudoku grid.
void printGrid() {
for (int r = 0; r < 9; r++) {
if (r % 3 == 0 && r != 0)
cout << "------+-------+------\n";

for (int c = 0; c < 9; c++) {
if (c % 3 == 0 && c != 0)
cout << "| ";

if (grid[r][c] == 0)
```

```
cout << "_ ";
else
cout << grid[r][c] << ' ';
}
cout << '\n';
}
}

// Recursive depth-first search with backtracking.
void solveRecursive() {
// Find the next empty cell.
int row = -1;
int col = -1;

for (int r = 0; r < 9; r++) {
for (int c = 0; c < 9; c++) {
if (grid[r][c] == 0) {
row = r;
col = c;
break;
}
}

if (row != -1)
break;
}

// No empty cells remain: a solution has been found.
if (row == -1) {
solutionCount++;
cout << "\nSolution " << solutionCount << ":\n";
printGrid();
return;
}

// Try every possible digit in this empty cell.
for (int num = 1; num <= 9; num++) {
if (isSafe(row, col, num)) {
// Make a choice.
grid[row][col] = num;

// Recursively search for further solutions.
solveRecursive();

// Backtrack: undo the choice and try another digit.
grid[row][col] = 0;
}
}
}
```

```
// Verify that the original clues do not conflict.
bool validInitialGrid() {
for (int r = 0; r < 9; r++) {
for (int c = 0; c < 9; c++) {
if (grid[r][c] != 0) {
int num = grid[r][c];

// Temporarily remove the clue so that
// isSafe checks for other matching clues.
grid[r][c] = 0;
bool valid = isSafe(r, c, num);
grid[r][c] = num;

if (!valid)
return false;
}
}
}

return true;
}

public:
SudokuSolver() {
solutionCount = 0;

for (int r = 0; r < 9; r++) {
for (int c = 0; c < 9; c++) {
grid[r][c] = 0;
}
}
}

// Read a puzzle containing digits and underscore blanks.
bool loadPuzzle(const string& filename) {
ifstream file(filename);

if (!file.is_open()) {
cout << "Error: Could not open " << filename << ".\n";
return false;
}

// Reset the grid before loading a new puzzle.
for (int r = 0; r < 9; r++) {
for (int c = 0; c < 9; c++) {
grid[r][c] = 0;
}
}

int cellCount = 0;
```

```
char ch;

// Read characters, ignoring spaces and line breaks.
while (file.get(ch)) {
if (ch >= '1' && ch <= '9') {
if (cellCount >= 81) {
cout << "Error: Invalid puzzle size in "
<< filename << ".\n";
return false;
}

grid[cellCount / 9][cellCount % 9] = ch - '0';
cellCount++;
}
else if (ch == '_') {
if (cellCount >= 81) {
cout << "Error: Invalid puzzle size in "
<< filename << ".\n";
return false;
}

grid[cellCount / 9][cellCount % 9] = 0;
cellCount++;
}
}

if (cellCount != 81) {
cout << "Error: " << filename
<< " must contain exactly 81 cells.\n";
return false;
}

return true;
}

// Display the original puzzle and find every solution.
void solve(const string& filename) {
cout << "\n====================================\n";
cout << "Puzzle file: " << filename << '\n';
cout << "====================================\n";

cout << "\nOriginal puzzle:\n";
printGrid();

solutionCount = 0;

if (!validInitialGrid()) {
cout << "\nNo solution found\n";
return;
}
```

```
solveRecursive();

if (solutionCount == 0) {
cout << "\nNo solution found\n";
}
else {
cout << "\nTotal solutions found: "
<< solutionCount << '\n';
}
}
};

int main() {
SudokuSolver solver;

// Process all five puzzle files.
for (int i = 1; i <= 5; i++) {
string filename = "puzzle" + to_string(i) + ".txt";

if (solver.loadPuzzle(filename)) {
solver.solve(filename);
}
}

return 0;
}
```

Section 4: Code Comparison

Correctness:

Both programs yield the same, correct results. Both programs are able to successfully read the provided files without error. Both programs provide cleanly-formatted outputs.

Execution Time:

Both programs use a brute-force, depth-first approach to solving the problem and thus have time complexity of O(k^n), with k = 9 as this is a Sudoku. ChatGPT's code contains many additional logic checks and branches than Gemini's within its helper functions, and is therefore likely slower to execute overall compared to Gemini. In particular, ChatGPT's method of loading a puzzle requires iterating though each character in the file and comparing it to characters, then converting ti to a number; whereas Gemini's simply feeds the characters in directly which executes notable faster.

Space Complexity:

Both programs act in-place, modifying an overall array with each recursion and thus having space complexity of O(1). Since these functions are recursive, they will have an additional impact on spaced used on the stack of O(n), where n is the number of unsolved spaces in the provided puzzle.

Maintainability:

Gemini's is clearly more maintainable than ChatGPT's. Gemini's program is both shorter – being nearly half the length – and features salient comments and easily followable names. Chat GPT's technically has more comments in absolute terms, but those comments are more spread-out through the lines of code.

Section 5: Selection

Gemini is the clearly superior program. It is faster overall compared to ChatGPT's, and will be easier to maintain relative to GPT's.

Section 6: Improvements

The only improvement I could make to the code as generated is to add line-by-line comments to marginally improve its maintainability. The generated code is close to optimal for the requirements of this assignment.